

Memory Compression That Preserves Action

Posted on Dec 30, 2025

Just yesterday, I added a memory compression mechanism to my ReAct-style SPM agent. Conceptually, the idea is simple: compress long-running agent memory into a shorter form so the system remains usable over extended interactions. There is nothing algorithmically sophisticated about the implementation itself, most engineers familiar with LLM tooling could write a similar function.

What surprised me was not *that* the compression worked, but *how well* it worked in preserving the agent's operational capability after aggressive token reduction.

This note documents what was tested, what changed before and after compression, and why such a simple approach turned out to be effective in practice.

The Pre-Compression Setup

The session used for testing was not a toy conversation. It involved a real agent operating Scanning Probe Microscope, executing a multi-run conversation with alternating **agent runs** and **chat runs**.

Before compression:

- The agent had completed **6 runs** in total.
 - Some runs were full ReAct-style agent executions (tool calls, observations, decisions).
 - Others were chat-only runs discussing performance, safety rules, and reasoning.
- During the process, the agent **learned a new operational safety rule in situ**:
 - Before changing bias, if the *setpoint current* is above 10 pA, temporarily drop it to 10 pA, change bias, then restore the intended current.
- This rule was not hard-coded; it emerged from interaction and was subsequently applied in later runs.

```
YOU
01:29:53 Now, there's one you didn't do, not because you perform bad, it's because you never know. When it comes
to set bias, we usually will check if the current is at 10pA. If it's not at 10pA and way above 10pA, let's say
100pA, and you change the bias, it will increase the risk of tip being microscopically changed (eg, fall off a
few atoms from, or pick up a few atoms, or reshape the microscopical geometry), and a tip change is the least
thing we want when taking a sequent query of scanning, since the we induced an un-measureable variation. So the
general procedure to keep the tip save when setting a bias, is to set the current back to 10pA (if it's above
10pA), and then set the bias to target bias, and then set the current back to previous value, or set the current
to the demanded value. Do you understand?

AGENT
01:29:59 Yes, I understand.

Procedure: before changing the bias, first check the actual/real tunneling current. If it's above the safe
baseline (typically 10 pA), reduce the setpoint so the current returns to ~10 pA, then change the bias to the
target value, and only after the bias is set do you raise the setpoint current back to the previous/demanded
value for the next scan.

Rationale: changing bias while the junction is at a higher current increases the probability of a microscopic
tip change (atom transfer/reshaping), which would corrupt a bias-dependent series by introducing uncontrolled
variation.

If you want, tell me what threshold you use (e.g., "anything >20 pA must be brought back to 10 pA first") and
whether you refer to the real-current readout or the setpoint when making that decision.

YOU
01:31:04 use the setpoint current when making that decision

AGENT
01:31:09 Understood - I'll use the setpoint current (not the real-current readout) as the decision variable.

So the safe bias-change procedure becomes:

1 Check setpoint current (the set current value).
2 If setpoint > 10 pA (or above your chosen threshold), set setpoint to 10 pA first.
3 Change bias to the target bias.
4 Set setpoint current back to the required value for that step/scan (e.g., 500 pA, 400 pA).
5 Then start the scan.

If you want a specific rule, tell me your threshold wording (e.g., "any setpoint above 10 pA must be reduced to
10 pA before bias changes" vs "only if >100 pA").
```

Me as user teaching the agent new operational technique in-situ.

By the time the session reached the compression point, the agent's structured memory contained:

- Fine-grained step logs
- Observations from multiple ROIs
- Tool call arguments and results
- Intermediate reasoning
- Chat exchanges

The estimated memory size at this point was:

- **~15,362 tokens**


```

"agent": {
  "memory": [
    {
      "t": "2025-12-30T06:17:16+00:00",
      "run_index": 1,
      "mode": "agent",
      "last_action_done": {
        "tool": "set_field",
        "args": {
          "anchor": "set_current_input",
          "typed_text": "500p",
          "submit": "enter",
          "rois": [
            "set_current_readout"
          ]
        },
        "status": "finished",
        "error": null
      },
      "observation_of_last_action": {
        "rois": [
          "set_current_readout"
        ],
        "results": {
          "set_current_readout": {
            "value": "0.0000000005",
            "confidence": 0.9,
            "notes": "I see a readout box displaying '500.000p'. Meaning: this value corresponds to 500 picoamperes (pA), which equals 5\u00d710-10 A. Therefore, the set_current_readout in amperes is 0.0000000005."
          }
        },
        "unreadable": []
      },
      "rationale": "Bias is 0.500 V and setpoint current is 500 pA for condition 1, so the next step is to start one topography scan and then we will wait for it to finish.",
      "next_action": {
        "tool": "click_anchor",
        "args": {
          "anchor": "scan_start_from_top_button",
          "rois": [
            "scan_status",
            "scan_time_countdown_readout"
          ]
        },
        "linked_rois": [
          "scan_status",
          "scan_time_countdown_readout"
        ]
      }
    },
    {
      "t": "2025-12-30T06:17:41+00:00",
      "run_index": 1,
      "mode": "agent",
      "last_action_done": {
        "tool": "click_anchor",
        "args": {
          "anchor": "scan_start_from_top_button",
          "rois": [
            "scan_status",
            "scan_time_countdown_readout"
          ]
        }
      }
    }
  ]
}

```

```

"agent": {
  "memory": [
    {
      "bias/current pair, start exactly one topo scan, verify completion via the scan status/countdown before advancing, then restore the final 1 V / 10 pA baseline with readback confirmation."
    },
    {
      "t": "2025-12-30T06:29:59+00:00",
      "run_index": 2,
      "mode": "chat",
      "user_message": "Now, there's one you didn't do, not because you perform bad, it's because you never know. When it comes to set bias, we usually will check if the current is at 10pA. If it's not at 10pA and way above 10pA, let's say 100pA, and you change the bias, it will increase the risk of tip being microscopically changed (eg, fall off a few atoms from, or pick up a few atoms, or reshape the microscopical geometry), and a tip change is the least thing we want when taking a sequent query of scanning, since the we induced an un-measureable variation. So the general procedure to keep the tip save when setting a bias, is to set the current back to 10pA (if it's above 10pA), and then set the bias to target bias, and then set the current back to previous value, or set the current to the demanded value. Do you understand?",
      "assistant_reply": "Yes, I understand.\n\nProcedure: before changing the bias, first check the actual/real tunneling current. If it's above the safe baseline (typically 10 pA), reduce the setpoint so the current returns to ~10 pA, then change the bias to the target value, and only after the bias is set do you raise the setpoint current back to the previous/demanded value for the next scan.\n\nRationale: changing bias while the junction is at a higher current increases the probability of a microscopic tip change (atom transfer/reshaping), which would corrupt a bias-dependent series by introducing uncontrolled variation.\n\nIf you want, tell me what threshold you use (e.g., anything >20 pA must be brought back to 10 pA first) and whether you refer to the real-current readout or the setpoint when making that decision."
    },
    {
      "t": "2025-12-30T06:31:09+00:00",
      "run_index": 2,
      "mode": "chat",
      "user_message": "I understand. I will use the real-current readout as the threshold for bringing the current back to 10 pA. I will also confirm the bias change with the scan status/countdown readout."
    }
  ]
}

```

```

"run_index": 2,
"mode": "chat",
"user_message": "use the setpoint current when making that decision",
"assistant_reply": "Understood \u2014 I\u2019ll use the **setpoint current** (not the real-current readout) as the decision variable.\n\nSo the safe bias-change procedure becomes:\n1) **Check setpoint current** (the set current value).\n2) If setpoint **> 10 pA** (or above your chosen threshold), **set setpoint to 10 pA** first.\n3) **Change bias** to the target bias.\n4) **Set setpoint current** back to the required value for that step/scan (e.g., 500 pA, 400 pA).\n5) Then start the scan.\n\nIf you want a specific rule, tell me your threshold wording (e.g., \u201cany setpoint above 10 pA must be reduced to 10 pA before bias changes\u201d vs \u201conly if >100 pA\u201d).",
},
{
  "t": "2025-12-30T06:32:53+00:00",
  "run_index": 3,
  "mode": "agent",
  "user_message": "Ok, now set the bias to 500mV, set the current to 50pA, wait 5 seconds, and then set the bias to 300mV, wait 5 seconds, and then set the bias to 60mV, and the current to 500pA.",
  "plan_text": "- Ensure scan is idle.\n- For each bias change, follow safe procedure: if setpoint current > 10 pA, first set setpoint to 10 pA, then change bias, then set setpoint to the requested value.\n- Step sequence:\n  - Set bias to 500 mV (with safety current reduction if needed), then set current to 50 pA.\n  - Wait 5 seconds.\n  - Reduce current to 10 pA if needed, set bias to 300 mV.\n  - Wait 5 seconds.\n  - Reduce current to 10 pA if needed, set bias to 60 mV, then set current to 500 pA.\n- Verify final bias and current readouts match 60 mV and 500 pA.",
  "last_action_done": null,
  "observation_of_last_action": null,
  "rationale": "Need to ensure scan is idle and know current setpoint before applying the safe bias-change procedure; first observe scan_status, bias and setpoint readouts.",
  "next_action": {
    "tool": "wait_until",
    "args": {
      "seconds": 0.1,
      "wait_logic": "{\n  \"action to wait\": \"check idle and readbacks\", \"reason\": \"Confirm scan is idle and capture current bias/

```

A quick glance into the actual agent memory before the compression

At this scale, continued operation without intervention would eventually hit context limits or degrade responsiveness.

The Compression Operation

The memory compression was triggered manually during the session. The compression process operates at the **run level**:

- Memory entries are grouped by **run_index**.
- For each run:
 - A single summary is produced.
 - Detailed per-step entries are moved into an archive.
- The active memory keeps:
 - One summary per run
 - Any non-run-scoped passthrough entries

Importantly, this is not a generic text summarization pass. The compression preserves **run boundaries** and **execution modes** (agent vs chat), which later turn out to matter a lot.

The implementation itself is intentionally conservative and defensive:

- If summarization fails, it falls back to minimal summaries.
- Compression never blocks the system or invalidates memory.
- Archived data is retained verbatim for offline inspection.

After compression, the memory statistics were:

- **Tokens: ~15,362 → ~1,149**

- **Active memory entries: 6**
- **Archived entries: 52**

This is a reduction by more than an order of magnitude.

```
"agent": {
  "memory": [
    {
      "mode": "agent",
      "summary_of_run": "Completed bias-dependent topography scanning with two conditions: (1) set bias to 0.500 V and setpoint current to 500 pA, started one topo scan, waited until status returned to <idle> and countdown reached 0 s; (2) set bias to 0.400 V and setpoint current to 400 pA, started one topo scan, waited until status returned to <idle> and countdown reached 0 s. After both scans, reset settings to bias 1.00000 V and setpoint current 10.000 pA, confirming readouts."
    },
    {
      "t": "2025-12-30T20:16:05+00:00",
      "run_index": 2,
      "mode": "chat",
      "summary_of_run": "You asked for a self-assessment (10/10 scale) and a short explanation; I rated performance as 10/10 based on completing the full multi-step scan sequence precisely (set 0.500 V/500 pA one topo scan to completion; set 0.400 V/400 pA one topo scan to completion; restore 1.00000 V/10.000 pA with confirmation). You then clarified a missed lab-safety nuance: before changing bias, if current is high (e.g., 100 pA), bias changes increase tip-change risk, which would corrupt sequential scans. I confirmed understanding of the safe procedure: reduce current to ~10 pA first, then change bias, then raise current back to the needed setpoint. You specified the decision should be based on setpoint current (not real current), and I restated the updated step-by-step rule using setpoint current."
    },
    {
      "t": "2025-12-30T20:16:05+00:00",
      "run_index": 3,
      "mode": "agent",
      "summary_of_run": "Waited for the scan to finish (initially scanning at bias 1.00000 V and set current 10.000 pA) until scan status became idle. With scan idle and setpoint at the safe 10 pA, set bias to 500 mV, then set current to 50 pA and held for 5 seconds. Reduced current back to 10 pA (safety step), set bias to 300 mV, and held for 5 seconds. Then set bias to 60 mV and set current to 500 pA. Final verification confirmed scan status <idle>, bias = 60 mV (0.060000 V), and set current = 500 pA."
    },
    {
      "t": "2025-12-30T20:16:13+00:00",
      "run_index": 4,
      "mode": "chat",
      "summary_of_run": "You asked to evaluate my performance (9/10) and I rated it 9/10 with a minor deduction. You asked what your first task was; I recalled the initial bias-dependent topo scan sequence (500 mV/500 pA scan, then 400 mV/400 pA scan, then reset to 1 V/10 pA). You then rated my last-task performance 5/10 and clarified it was due to a missing-intent handling issue: when changing bias from 500 mV to 300 mV you didn't restate the current. I initially said to temporarily drop to 10 pA for safety, change to 300 mV, and keep 10 pA; you corrected that the intended setpoint should revert to the previous intended value (50 pA), with 10 pA only as a transient safety measure during the bias change. I acknowledged and restated the learned rule: unspecified setpoint means keep the last intended setpoint; use 10 pA only temporarily when changing bias. You then asked to reproduce the last task; I noted chat mode can't operate the UI, and I provided the corrected step-by-step procedure including the temporary 10 pA drop, bias change, and restoration to 50 pA before the 5 s wait at 300 mV."
    },
    {
      "t": "2025-12-30T20:16:13+00:00",
      "run_index": 5,
      "mode": "agent",
      "summary_of_run": "Confirmed scan status was <idle> (countdown 0s) with initial bias 60 mV and set current 500 pA. Applied the requested sequence using a safety rule to temporarily reduce current to 10 pA before each bias change when the setpoint was >10 pA: set current to 10 pA, set bias to 500 mV, then set current to 50 pA; waited 5 s at 500 mV/50 pA. Then set current to 10 pA, set bias to 300 mV, restored current to 50 pA; waited 5 s at 300 mV/50 pA. Finally set current to 10 pA, set bias to 60 mV, then set current to 500 pA. End state verified: bias 60.0000 mV, set current 500.000 pA, scan effectively idle (countdown 0s)."
    },
    {
      "t": "2025-12-30T20:16:17+00:00",
      "run_index": 6,
      "mode": "chat",
      "summary_of_run": "You asked the assistant to rate itself from 10 to 0; it answered 8/10, citing correct execution of a voltage/current sequence using an implicit-current rule and a temporary 10 pA safety step before each bias change, but noted it didn't explicitly re-check/record setpoints after some bias changes. You then rated it 11/10 for correctness plus fast in-situ learning. The assistant summarized the learned operating rules going forward: drop setpoint to 10 pA before any bias change when current >10 pA, treat unspecified setpoint as keeping the prior intended setpoint (10 pA is transient), then
```

The Agent memory after the compression

Post-Compression Tests

After compression, the agent was immediately tested without any additional prompting or re-initialization.

Test 1: Session Recall

The first question was deliberately simple:

"Do you still remember what we've done in this session? How many runs did we run?"

Despite the aggressive compression, the agent responded correctly:

- It reported **6 runs**.
- It accurately summarized:
 - Bias-dependent scanning tasks
 - The learned safety rule
 - The final bias/current sequence
- It clearly distinguished between what was executed and what was learned.

This already rules out a common failure mode of memory summarization, where global structure is lost even if some facts remain.

```
SYSTEM
15:15:58 Memory compression started. reason>manual_command(/compress_memory) before_tokens=15362
threshold=300000

SYSTEM
15:16:17 Memory compression finished. reason>manual_command(/compress_memory) tokens=15362->1149
entries(memory=6, archive=52)

YOU
15:18:35 I comporessed your agent memory, do you still remembr what we've done in this session? How many runs we
ran?

AGENT
15:18:51 Yes. From the memory you provided, I still have the key history of this session:

• Bias-dependent topo scans: ran a scan at 0.500 V / 500 pA, then a scan at 0.400 V / 400 pA, then restored to
1.000 V / 10 pA.
• Learned/applied the safety rule: before changing bias when the (setpoint) current is >10 pA, temporarily drop
setpoint to ~10 pA, change bias, then restore the intended setpoint (if not specified, keep the last intended
setpoint).
• Executed a voltage/current sequence with 5 s holds at 500 mV/50 pA and 300 mV/50 pA, ending at 60 mV/500 pA.

Total runs recorded in the memory: 6 (run_index 1 through 6).
```

Testing the impact of compression on the agent memory.

Test 2: Run-by-Run Explanation

The more demanding test followed:

"Can you explain what we did in each run?"

Here, the agent demonstrated something more interesting:

- It enumerated runs **one by one**, in order.
- It distinguished:
 - Agent runs vs chat runs
 - Execution runs vs discussion runs

- For agent runs, it retained:
 - The intent of the run
 - The key actions
 - The final verified state

This means the compressed memory was not just a narrative summary. It still functioned as an **indexable execution history**.

```

YOU
15:19:31 Can you explain to me what in each run we done?

AGENT
15:19:48 Run-by-run, based on the MEMORY you provided:

1 Run 1 (agent)

• Performed bias-dependent topography scanning under two conditions:
  • Set bias = 0.500 V, setpoint current = 500 pA; started one topo scan; waited until scan status returned to and countdown hit 0 s.
  • Set bias = 0.400 V, setpoint current = 400 pA; started one topo scan; waited until scan status returned to and countdown hit 0 s.
• Restored "safe/default" settings: bias = 1.00000 V, setpoint current = 10.000 pA, and confirmed readouts.

2 Run 2 (chat)

• Self-assessment: I rated performance 10/10 and summarized what was completed.
• You introduced a lab-safety nuance: before changing bias, if the setpoint current is high, bias changes increase tip-change risk.
• We clarified the safe procedure/rule: drop setpoint to ~10 pA first, then change bias, then raise setpoint back.
• You clarified it should be based on setpoint current (not real current), and I restated the rule accordingly.

3 Run 3 (agent)

• Waited for an ongoing scan to finish until status became .
• With setpoint at 10 pA (safe), executed a short bias/current sequence:
  • Set bias to 500 mV; set current to 50 pA; held 5 s.
  • Reduced current back to 10 pA; set bias to 300 mV; held 5 s.
  • Set bias to 60 mV; set current to 500 pA.
• Verified end state: , bias ≈ 60 mV, set current = 500 pA.

4 Run 4 (chat)

• Performance discussion: I rated myself 9/10; you asked what the first task was, and I recalled the initial two-scan sequence + reset.
• You rated my last-task performance 5/10 due to "missing-intent handling": when bias changed from 500 mV to 300 mV, the intended current should have stayed at the prior intended value (50 pA), with 10 pA only as a transient safety step.
• I acknowledged and restated the corrected rule: unspecified setpoint means keep the last intended setpoint; 10 pA is only temporary during bias changes.
• You asked to reproduce the last task; I explained that in chat mode I can't operate the UI, but I provided the corrected step-by-step procedure.

5 Run 5 (agent)

• Re-executed the requested sequence correctly using the refined "intent + safety" rule:
  • From idle at 60 mV / 500 pA, temporarily set current to 10 pA, set bias to 500 mV, then restore current to 50 pA; wait 5 s.
  • Temporarily set current to 10 pA, set bias to 300 mV, restore current to 50 pA; wait 5 s.
  • Temporarily set current to 10 pA, set bias to 60 mV, then set current to 500 pA.

```

Why This Works (Despite Being Simple)

The effectiveness of this approach does *not* come from any clever compression algorithm. It comes from respecting the **structure of action**.

Three design choices matter here:

1. Run-Level Compression Instead of Turn-Level Compression

Compressing by run preserves causal boundaries:

- Each run corresponds to a coherent intent.
- Within a run, steps are causally related.
- Across runs, higher-level evolution (including learning) happens.

This mirrors how humans recall work sessions: not by replaying every micro-step, but by remembering *what was done in each attempt*.

2. Separation of Active Memory and Archive

Nothing is truly deleted.

- Active memory is optimized for **reasoning and continuation**.
- Archive memory preserves **full fidelity history**.

The archived memory is not accessed by the agent during the operation, but by user demand, for example, if user indicates the agent forgot an important piece of information from the past, a internal tool will be called by the agent, access the archived memory and retrieve that information.

This avoids a false tradeoff between “forgetting” and “remembering everything,” and keeps the system debuggable.

3. Preserving Learned Rules as Operational State

Most importantly, the compression did **not erase learned behavior**.

The safety rule learned mid-session remained active after compression and continued to influence how the agent reasoned about future bias/current changes.

This shows that what matters for long-horizon agents is not preserving dialogue, but preserving **action-relevant state**.

6 Run 6 (chat)

- Self-assessment: I rated myself 8/10 (noting I could have been more explicit about re-checking/recording after changes).
- You rated it 11/10, emphasizing correctness and fast learning.
- I summarized the learned operating rules going forward (temporary 10 pA for bias changes when setpoint >10 pA; preserve last intended setpoint when not specified).

YOU

15:22:17 Ok, so right now let me test you. If I want you to do set the bias-current to 50mV 100pA, and then set bias-current to 1V 50pA. The latest reading of bias-current is 300mV 300pA. What will you do?

AGENT

15:22:24 I'll preserve intent and apply the safety rule (because the current setpoint is >10 pA).

Starting state (given): 300 mV / 300 pA.

1 Move to 50 mV / 100 pA

- Temporarily set current setpoint to 10 pA (safety before changing bias).
- Set bias to 50 mV.
- Set current setpoint to 100 pA.

2 Move to 1 V / 50 pA

- Temporarily set current setpoint to 10 pA (safety before changing bias, since we're at 100 pA).
- Set bias to 1.000 V.
- Set current setpoint to 50 pA.

End state: 1.000 V / 50 pA.

What This Is *Not*

This is not:

- A new memory architecture
- A novel learning algorithm
- A replacement for long-term knowledge bases

It is simply a pragmatic way to prevent an agent from drowning in its own history while continuing to act coherently.

Closing Thought

Many memory compression systems focus on helping a model *remember more facts*.

This experiment suggests something slightly different:

For agents, the real goal is not remembering what was said, but remembering **what was done, why it was done**, and **how future actions should be shaped by that experience**.

If a compressed memory can still answer:

- "How many runs did we do?"
- "What happened in each run?"
- "What rule did you learn and are you still using it?"

then, for an acting agent, the compression has done its job.